Step 2 — A Real ReAct Agent That Calls a Discovered Tool — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

This is the standard tool-calling loop from Doc07 / Project 2 — the only difference is where the tools come from. Two small adapters sit between MCP and OpenAI, and the loop uses them.

- **The schema bridge:** OpenAI's `tools` wants `{"type": "function", "function": {"name", "description", "parameters"}}`. An MCP tool's `input_schema` is already JSON Schema, so `parameters` is just that schema copied in. Guard the `None` description → `""`.
- **The result extractor:** `await session.call_tool(name, args)` returns a `CallToolResult` whose `.content` is a list of content blocks (`TextContent` , with `.text`) — not a string. Join the `.text` of each block.
- **The loop:** `ask_model` returns a message with `.content` and `.tool_calls`. If there are no tool calls, that's the final answer. Otherwise, for each call read `call.function.name` and `json.loads(call.function.arguments)`, dispatch it, and append a `{"role": "tool", "tool_call_id": call.id, "content": observation}` message. Repeat.
- **The cap:** a plain `for step in range(max_iterations)` — Doc07's "a limit isn't optional." Raise `MaxIterationsExceeded` if it runs out.

Give the pool two new methods — `openai_tools()` and `call_tool(namespaced_name, args)` — so the loop never touches a session directly.

Hint 2 — The plan, and almost the whole thing

```
tool_bridge.py:
    mcp_tools_to_openai_schema(named_tools):   # list of (namespaced, tool)
        each -> {"type":"function",
                 "function":{name, description or "",
                             parameters: tool.input_schema}}
    extract_text(call_result):
```

```
        join block.text for each block in call_result.content

server_pool.py  (+ two methods):
    openai_tools(): return mcp_tools_to_openai_schema(self.named_tools)
    async call_tool(namespaced, args):
        server, tool = self.routing[namespaced]
        result = await self.sessions[server].call_tool(tool, args)
        return extract_text(result)

agent.py:
    async run_agent(task, pool, client, max_iterations=6):
        messages = [system, user=task]
        tools = pool.openai_tools()
        for step in range(max_iterations):
            reply = ask_model(client, messages, tools)
            if not reply.tool_calls: return AgentResult(reply.content, steps)
            append assistant message (with tool_calls as dicts)
            for call in reply.tool_calls:
                args = json.loads(call.function.arguments)
                obs = await pool.call_tool(call.function.name, args)
                append {"role":"tool","tool_call_id":call.id,"content":obs}
        raise MaxIterationsExceeded(...)
```

The one subtlety: when you append the assistant turn that made tool calls, it must carry those `tool_calls` back in the message (as plain dicts with `id` / `type` / `function`), or the follow-up `tool` messages have nothing to attach to and the API rejects the next call. The solution's `_tool_calls_as_dicts` helper does exactly that.

The whole file set, runnable, is in the Solution.